

Solving the N-Queens Problem with Exhaustive Search and Genetic Algorithms

Akhil Nair

Department of Business

AI Research Group

Univ. of Europe for Applied Sciences

Konrad-Ruse Ring 11, 14469 Potsdam, Germany

akhil.nair@ue-germany.de

Raja Hashim Ali

Department of Business

AI Research Group

Univ. of Europe for Applied Sciences

Konrad-Ruse Ring 11, 14469 Potsdam, Germany

hashim.ali@ue-germany.de

Abstract—The N-Queens problem is a well-known problem in artificial intelligence and optimization research. The objective is to place N queens on an $N \times N$ chessboard in such a way that no two queens attack each other horizontally, vertically, or diagonally. In this project, four different approaches were implemented and compared for solving the N-Queens problem: Exhaustive Depth-First Search (DFS), Hill Climbing, Simulated Annealing, and Genetic Algorithms. The algorithms were implemented in Python and tested for board sizes $N = 10, 30, 50, 100, 200$, and 500 . The performance of each algorithm was analyzed using runtime and memory usage. The results showed that exhaustive search methods become difficult to use for larger board sizes, while optimization-based methods scale better but may not always reach the perfect solution. DFS produced exact solutions for smaller board sizes but became computationally expensive for larger values of N . Hill Climbing reduced runtime significantly but often got trapped in local optima. Simulated Annealing improved exploration by occasionally accepting worse states, while Genetic Algorithms explored multiple candidate solutions simultaneously using evolutionary operations. Overall, optimization-based approaches performed better for larger board sizes, while exhaustive search remained useful for smaller exact search problems.

Index Terms—N-Queens problem, optimization algorithms, genetic algorithm, exhaustive search technique, local search techniques

I. INTRODUCTION

The N-Queens problem is one of the most widely studied problems in artificial intelligence, optimization, and constraint satisfaction systems [1], [2].

The problem requires placing N queens on an $N \times N$ chessboard such that no two queens threaten each other horizontally, vertically, or diagonally.

As the value of N increases, the number of possible configurations grows very rapidly, making exhaustive search approaches computationally expensive.

In this project, four different algorithms were implemented and compared:

- Exhaustive Depth-First Search (DFS)
- Greedy Hill Climbing
- Simulated Annealing
- Genetic Algorithms

The algorithms were compared using runtime, memory usage, scalability, and solution behavior.

A. Chessboard Constraints

In the N-Queens problem, each queen must be placed in a position where it does not attack any other queen on the board. A queen can attack horizontally, vertically, and diagonally. Therefore, a valid solution must satisfy all three constraints simultaneously.

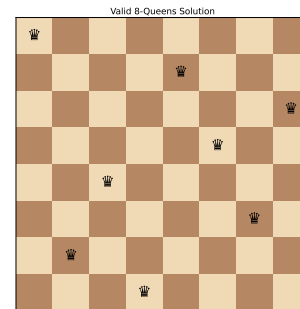


Fig. 1. Example representation of the N-Queens problem

The complexity of the problem increases rapidly as the value of N becomes larger because the number of possible board configurations grows exponentially. This makes the N-Queens problem an important benchmark for evaluating search and optimization algorithms.

Figure 1 illustrates an example of a valid N-Queens configuration.

II. LITERATURE REVIEW

Depth-First Search and backtracking algorithms are commonly used for constraint satisfaction problems [1].

Hill Climbing and Simulated Annealing are widely used local search techniques in optimization problems [3], [4].

Genetic Algorithms are evolutionary optimization techniques inspired by biological evolution [5], [6].

The N-Queens problem is frequently used as a benchmark problem because it contains a large search space and strict constraints.

Traditional exhaustive search methods guarantee complete exploration of the search space but become computationally infeasible for larger board sizes.

Optimization-based approaches attempt to reduce runtime and improve scalability using heuristic and probabilistic techniques.

A. Why Multiple Algorithms Were Compared

Different algorithms solve the N-Queens problem using different strategies. Exhaustive search methods attempt to explore all possible configurations systematically, while heuristic and optimization-based methods attempt to find acceptable solutions faster by reducing the search space.

The purpose of comparing multiple approaches in this project was to analyze the trade-off between solution accuracy, runtime, memory usage, and scalability for large board sizes.

Recent studies have compared classical exhaustive search methods and metaheuristic optimization approaches for solving the N-Queens problem efficiently [2], [7], [8].

III. METHODOLOGY

A. Experimental Environment

All experiments were performed using Python 3.13 on a Windows-based system. Runtime measurements were recorded using Python timing libraries, while memory consumption was analyzed using memory tracing tools. The generated workflow diagrams and result visualizations were created using Graphviz and Matplotlib. The experiments were repeated multiple times to observe the behavior of different algorithms for increasing board sizes.

B. Problem Representation

The board was represented using a one-dimensional array where each index represented a row and the value represented the column position of a queen.

For example:

$$[1, 3, 0, 2]$$

represents a valid 4-Queens configuration.

This representation simplifies the implementation because each row contains only one queen. Therefore, the algorithm only needs to determine the column positions while automatically avoiding horizontal conflicts. The one-dimensional representation also reduces memory usage compared to storing the complete chessboard matrix.

C. Exhaustive Depth-First Search (DFS)

DFS was implemented using recursive backtracking. The algorithm places queens row by row and checks whether each position is safe. The search continues recursively until either a valid solution is found or all possible configurations are exhausted. If a conflict occurs, the algorithm backtracks and tries a different queen placement.

Figure 2 shows the workflow used for the DFS implementation.

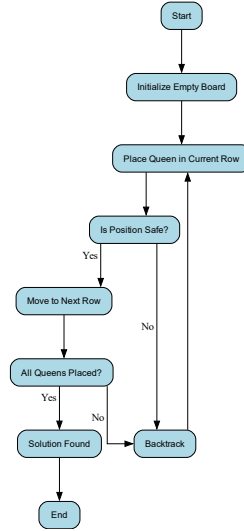


Fig. 2. Workflow of the Depth-First Search algorithm

D. Greedy Hill Climbing

Hill Climbing was implemented as a local search algorithm that attempts to reduce conflicts by moving queens to better neighboring positions [4].

Figure 3 shows the workflow used for the Hill Climbing implementation.

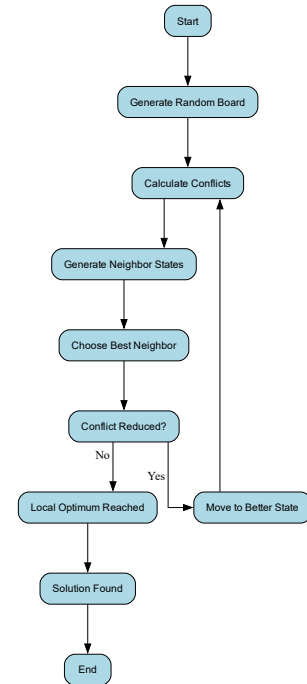


Fig. 3. Workflow of the Hill Climbing algorithm

E. Simulated Annealing

Simulated Annealing was implemented to overcome the local optimum limitations of Hill Climbing [3].

The probability function used was:

$$P = \min(1, e^{-\Delta E/T})$$

Figure 4 shows the workflow used for the Simulated Annealing implementation.

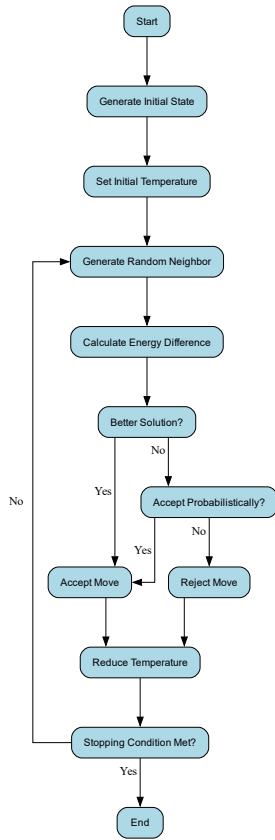


Fig. 4. Workflow of the Simulated Annealing algorithm

F. Genetic Algorithm

The Genetic Algorithm used population-based optimization with selection, crossover, and mutation operations [5], [9].

Figure 5 shows the workflow used for the Genetic Algorithm implementation.

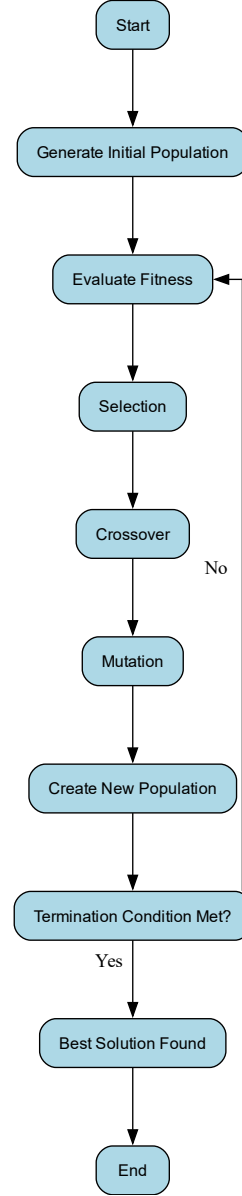


Fig. 5. Workflow of the Genetic Algorithm for solving the N-Queens problem

G. Evaluation Metrics

The algorithms were evaluated using the following criteria:

- Execution time
- Memory consumption
- Scalability for large board sizes
- Ability to reach valid solutions

These metrics were selected because they provide a good comparison between exhaustive search techniques and optimization-based approaches. Several recent studies have also highlighted the scalability advantages of stochastic op-

timization techniques over deterministic exhaustive search approaches for large-scale N-Queens instances [10].

IV. RESULTS AND ANALYSIS

A. Depth-First Search Results

TABLE I
DFS EXPERIMENTAL RESULTS

N	Status	Time (s)	Memory (MB)
10	Solved	0.0033	0.0039
30	Timeout	10.0438	0.0044
50	Timeout	10.0348	0.0052
100	Infeasible	–	–
200	Infeasible	–	–
500	Infeasible	–	–

The DFS algorithm successfully solved smaller board sizes but became computationally infeasible for larger values of N. Since DFS explores the search space recursively, the execution time increased rapidly with increasing board size. For N = 100 and above, the algorithm became impractical because of the exponential growth in possible configurations.

B. Greedy Hill Climbing Results

TABLE II
HILL CLIMBING EXPERIMENTAL RESULTS

N	Solved	Steps	Time (s)	Memory (MB)
10	False	1000	0.1297	0.0047
30	False	1000	1.3771	0.0018
50	False	1000	3.8248	0.0022
100	False	1000	15.1084	0.0050
200	False	1000	59.1300	0.0065
500	False	1000	615.0204	0.0259

Hill Climbing achieved significantly lower runtime compared to DFS because it used local search instead of exhaustive exploration. However, the algorithm frequently became trapped in local optima and was unable to consistently reach valid solutions for larger board sizes.

C. Simulated Annealing Results

TABLE III
SIMULATED ANNEALING EXPERIMENTAL RESULTS

N	Solved	Iterations	Time (s)	Memory (MB)
10	False	2297	0.0813	0.0012
30	False	2297	0.9126	0.0034
50	False	2297	2.7729	0.0052
100	False	2297	11.8371	0.0044
200	False	2297	49.3129	0.0081
500	False	2297	532.4400	0.0245

Simulated Annealing improved exploration by probabilistically accepting worse states during the optimization process.

This allowed the algorithm to escape local optima more effectively than Hill Climbing. The runtime increased gradually with larger board sizes, although the algorithm maintained relatively low memory usage.

D. Genetic Algorithm Results

TABLE IV
GENETIC ALGORITHM EXPERIMENTAL RESULTS

N	Solved	Generations	Time (s)	Memory (MB)
10	False	200	1.4124	0.0106
30	False	200	15.6479	0.0153
50	False	200	47.0435	0.0245
100	False	200	203.5354	0.0492
200	False	200	833.6867	0.0939
500	Terminated	–	–	–

The Genetic Algorithm explored multiple candidate solutions simultaneously using population-based optimization. Although the approach scaled better for larger board sizes compared to DFS, the repeated fitness evaluation and crossover operations resulted in higher runtime and memory consumption.

E. Comparative Analysis

During implementation, it was observed that the DFS approach becomes extremely slow even for medium-sized boards because the number of possible states grows rapidly.

TABLE V
ALGORITHM COMPARISON SUMMARY

Algorithm	Scalability	Runtime	Solution Quality
DFS	Low	Very High	Exact
Hill Climbing	Medium	Low	May get stuck
Simulated Annealing	High	Medium	Better exploration
Genetic Algorithm	High	High	Population-based

Hill Climbing produced faster results but often stopped before reaching an optimal configuration because of local optima.

Simulated Annealing improved exploration by occasionally accepting worse states during the search process.

Genetic Algorithms handled larger board sizes better than DFS, although the runtime increased because of repeated population evaluation.

Among all implemented approaches, Simulated Annealing provided a good balance between runtime and memory usage for medium-sized boards.

The runtime graph clearly shows that DFS grows rapidly with increasing board size. Simulated Annealing and Hill Climbing required significantly less memory and runtime compared to exhaustive search.

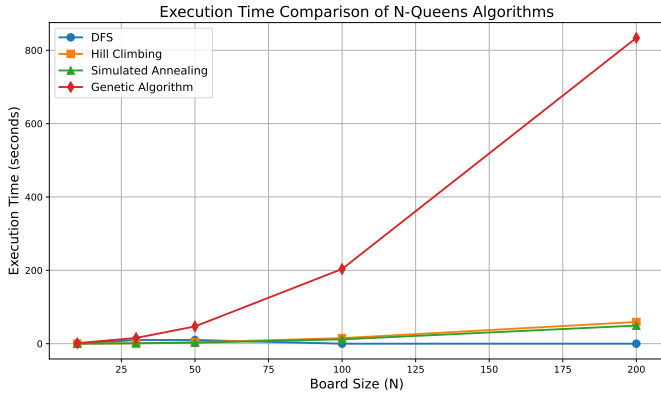


Fig. 6. Runtime comparison of the four algorithms

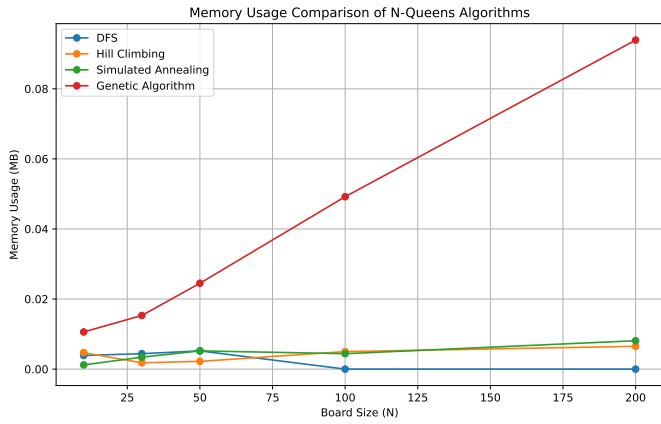


Fig. 7. Memory usage comparison of the four algorithms

F. Implementation Challenges

During implementation, one of the main challenges was handling the rapidly increasing search space for large board sizes. Exhaustive search approaches required significantly more computational time, while optimization-based algorithms required careful parameter tuning to improve solution quality and convergence behavior.

G. Advantages and Limitations

Each algorithm demonstrated different strengths and limitations during implementation.

DFS guaranteed complete exploration of the search space but suffered from exponential complexity for large board sizes.

Hill Climbing provided faster runtime performance but frequently became trapped in local optima.

Simulated Annealing improved exploration and scalability by accepting worse states probabilistically during the search process.

Genetic Algorithms explored multiple candidate solutions simultaneously and handled larger board sizes more effectively, although they required higher computational overhead because of repeated population evaluation.

V. CONCLUSION

This project compared four different approaches for solving the N-Queens problem: Depth-First Search, Hill Climbing, Simulated Annealing, and Genetic Algorithms.

The experimental results showed that DFS can find exact solutions for smaller board sizes, but its runtime increases very quickly as the board size grows.

Hill Climbing reduced runtime considerably, although it often became stuck in local optima.

Simulated Annealing improved exploration by occasionally accepting worse states during the search process.

Genetic Algorithms provided better scalability for larger board sizes, but they also required higher computational cost because of repeated population operations.

Overall, optimization-based methods performed better for larger N-Queens instances, while exhaustive search remained useful for smaller problem sizes where exact solutions are required.

REFERENCES

- [1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*. Prentice Hall, 2010.
- [2] G. Alizadehbirjandi, R. H. Ali, R. Koutaly, T. A. Khan, and I. Ahmad, "Solving n-queens problem using exhaustive search and a novel genetic algorithm," in *2025 International Conference on Innovation in Artificial Intelligence and Internet of Things (AIIT)*, 2025, pp. 1–6.
- [3] E. Aarts and J. Korst, *Simulated Annealing and Boltzmann Machines*. Wiley, 1988.
- [4] B. Selman, "Hill-climbing search," *Encyclopedia of Cognitive Science*, 1997.
- [5] D. E. Goldberg, *Genetic Algorithms in Search, Optimization and Machine Learning*. Addison-Wesley, 1989.
- [6] U. Sarkar and S. Nag, "An adaptive genetic algorithm for solving n-queens problem," *arXiv preprint arXiv:1802.02006*, 2017.
- [7] D. Carlberg, A. Raza, R. H. Ali, T. A. Khan, and S. Faiz, "A comparative analysis of the algorithms dfs, hill climbing, simulated annealing, and genetic algorithm for solving the n-queens problem," in *2025 27th International Multitopic Conference (INMIC)*, 2025, pp. 1–6.
- [8] L. Mikhailova, S. G. Abbas, M. Imad, and R. H. Ali, "Solving the n-queens puzzle: A performance study of classical and metaheuristic algorithms," in *2025 International Conference on Frontiers of Information Technology (FIT)*, 2025, pp. 1–6.
- [9] S. Sharma and V. Jain, "Solving n-queen problem by genetic algorithm using novel mutation operator," *IOP Conference Series: Materials Science and Engineering*, vol. 1116, no. 1, p. 012195, 2021.
- [10] D. Carlberg, A. Raza, R. H. Ali, T. A. Khan, and S. Faiz, "Evaluating deterministic and stochastic strategies for large-scale n-queens solutions," in *2025 International Conference on Frontiers of Information Technology (FIT)*, 2025, pp. 1–6.